

Gustina saobraćaja na graničnim prelazima

Klasifikacija slika sa kamera u tri klase gustine: AutoML optimizacija hiperparametara, offline evaluacija i serviranje kvantizovanog modela

Agenda

01 Problem i podaci

9.869 anotiranih kadrova sa 20 kamera-smerova

03 Implementacija i rezultati

Bajesova pretraga, osnovne linije, perturbacioni testovi

02 Teorija: treniranje i evaluacija

Aktuelno stanje u AutoML-u i offline evaluaciji

04 Primena i serviranje

Kvantizacija i REST API u kontejneru

PROBLEM

Problem i skup podataka

Jedan kadar sa kamere graničnog prelaza se klasifikuje u jednu od tri klase. Trening slike su ručno anotirane u četiri klase koje su u modelu spojene u tri, jer je granica low / moderate subjektivna.

no_traffic

4.071

light

4.005

high

1.793

Metapodaci (kamera, smer, vreme) se ne koriste kao podaci nad kojima model trenira klasifikator. Ubacivanjem ovih podataka bi učinilo model manje robusnim i naučio bi da klasifikuje slike na osnovu granica i vremena umesto auta na slici,



Vremenska podela podataka

TRAIN

$\leq 31.05.2024.$

6.080

18% klase high

VAL

06 $\rightarrow$ 12.2024.

2.378

24% klase high

TEST

$\geq 01.01.2025.$

1.411

7% klase high

Iz ovog razloga, jer su klase nebalansirane, metrika koja je izabrana je **macro F1**.

Treniranje i evaluacija modela

Aktuelno stanje u dve oblasti: automatizovana pretraga
hiperparametara i offline evaluacija

Pre početka treniranja

Referentna osnova - macro F1

Zero-rule

0.1887

Random (distr)

0.3224

Train - Val - Test

Vremenski podeljeno, zbog prirode podataka.

Izbor modela

Zamrznuti backbone - MobileNetV2, čiji je izlaz feature vektor.

Feature vektor je ulaz klasifikacione glave koja je trenirana pomoću AutoML-a.

AutoML - svrha

Šta je AutoML?

AutoML predstavlja praksu automatizacije procesa developmenta ML modela. On daje mogućnost ne-ML inženjerima da prave ML modele, kao i ML inženjerima da automatizuju proste delove pravljenja modela.

Kako funkcioniše?

Pomoću AutoML-a se pravi nekoliko pipeline-ova koji se paralelno testiraju i time je moguće identifikovati optimalni izbor. AutoML omogućava automatizaciju svih delova ML pipeline-a.

Limitacije AutoML-a

Brz porast cene treniranja, manjak interpretabilnosti, rizik od overfitinga, ograničena kontrola,...

AutoML - pregled oblasti

Optimizacija hiperparametara

HPO

Pretraga po mreži, nasumična pretraga, Bajesova optimizacija (GP, TPE, SMAC), bandit-metodi sa ranim prekidanjem (Hyperband).

korišćeno
u radu

Pretraga arhitekture

NAS

Od pojačanog učenja (NASNet) ka diferencijabilnoj pretrazi (DARTS) i predikcijama bez treniranja (zero-cost proxy). Skupo; retko opravdano kada postoji dobra prethodno trenirana mreža.

Meta-učenje

warm start

Prenos znanja sa prethodnih zadataka kao početna tačka pretrage (auto-sklearn 2.0, OpenML portfolija). Traži portfolio srodnih zadataka.

Bajesova optimizacija hiperparametara

01

Nasumična ili strateška inicijalizacija hiperparametarskog prostora

02

Aproksimacija ciljne funkcije korišćenjem Gausovog procesa i podataka iz 1. koraka

03

Određivanje sledećeg uzorkovanja u prostoru hiperparametara

03

Dobijanje rezultata na novim hiperparametrima i ponavljanje koraka 2, 4.

PREDNOST

Manje pokušaja za isti rezultat kada je funkcija cilja glatka i skupa za izračunavanje.

Offline evaluacija - metode

Pomeranje od jednog zbirnog broja ka evaluaciji kao skupu provera: gde model greši, na kom podskupu, i pod kojim uslovima ulaza.

Referentne osnove

Nulto pravilo i nasumična po distribuciji klasa. Bez njih metrika nema značenje, naročito uz neizbalansirane klase.

Evaluacija po podskupovima

Agregat krije podgrupe sa slabim rezultatom. Automatsko otkrivanje kritičnih preseka (SliceFinder, Model Cards).

Perturbacioni testovi

Invarijantnost na transformacije koje ne menjaju labelu, šum, osvetljenje, kompresija. Robusnost kao merena osobina, ne pretpostavka.

Kalibracija i saglasnost

Da li verovatnoća znači ono što tvrdi, i koliko se dve verzije modela slažu po pojedinačnoj predikciji — ne samo u agregatu.

Trendovi i alati

01	Multi-fidelity pretraga	Rano prekidanje slabih pokušaja (ASHA, BOHB) danas je podrazumevano, a ne dodatak.
02	Evaluacija kao test	Provere po podskupovima i robusnost ulaze u CI; prolaz postaje uslov za promociju modela.
03	Pomeranje fokusa	Uz jake prethodno trenirane mreže, dobitak se seli sa arhitekture na ulaznu reprezentaciju i način adaptacije.
04	Alati	Optuna, KerasTuner, Ray Tune, auto-sklearn; praćenje pokušaja kroz MLflow ili W&B.

Implementacija

KerasTuner Bayesian optimization nad klasifikacionom glavom, 25 pokušaja, nad keširanim karakteristikama zamrznute osnovne mreže.

```
src/training/train.py
```

Arhitektura rešenja

01 · prepare

Ekstrakcija feature-a

ROI maska, letterbox, jedan prolaz kroz zamrznutu mrežu.

02 · train

Bajesova pretraga

25 pokušaja nad glavom. Pokušaj traje reda par sekundi jer su karakteristike iste u svakoj epohi.

03 · evaluate

Offline evaluacija

Referentne osnova, matrica konfuzije, perturbacije.

04 · serve

Kvantizacija i REST

TFLite int8 u Docker kontejneru, izložen kao FastAPI endpoint.

~20-30×

manje računanja po epohi kada je osnovna mreža zamrznuta i karakteristike keširane bez toga pretraga od 25 pokušaja nije izvodljiva na laptopu.

Kod - učitavanje slika

src/serve/prepare.py



```
def load_image(path: str, size: int, roi=None, letterbox=False) -> np.ndarray:

    img = Image.open(path).convert("RGB")
    w, h = img.size
    arr = np.array(img, dtype=np.uint8)    # np.asarray on a PIL image is read-only
    for y0, y1, x0, x1 in OVERLAY_BOXES:
        arr[int(y0 * h):int(y1 * h), int(x0 * w):int(x1 * w)] = 0
    img = Image.fromarray(arr)
    if roi is not None:
        img = apply_roi(img, roi, letterbox=letterbox)
    return np.asarray(img.resize((size, size), Image.BILINEAR), dtype=np.float32)
```

Prostor pretrage

pool	global average pooling · flatten
n_layers	0 – 2
units	64 – 512, korak 64
dropout	0,0 – 0,6
lr	1e-4 – 1e-2, log

najbolja konfiguracija

```
pool = flatten n_layers = 2 → 128
dropout = 0,0 / 0,5 lr = 2,6e-3
```

Karakteristike zadržavaju prostornu mrežu 3×3 umesto globalnog sažimanja, pa gruba raspodela vozila u kadru preživi.

Kod — deklaracija prostora i pretraga

src/training/train.py

```
def build(hp):
    inp = layers.Input(shape=feat_shape)
    if hp.Choice("pool", ["gap", "flatten"]) == "gap":
        x = layers.GlobalAveragePooling2D()(inp)
    else:
        x = layers.Flatten()(inp)          # keeps coarse spatial layout
    x = layers.BatchNormalization()(x)
    for i in range(hp.Int("n_layers", 0, 2)):
        x = layers.Dense(hp.Int(f"units_{i}", 64, 512, step=64), activation="relu")
    (x) x = layers.Dropout(hp.Float(f"dropout_{i}", 0.0, 0.6, step=0.1))(x)
    out = layers.Dense(n_classes, activation="softmax")(x)

    model = keras.Model(inp, out)
    model.compile(
        optimizer=keras.optimizers.Adam(
            hp.Float("lr", 1e-4, 1e-2, sampling="log")),
        loss="categorical_crossentropy",
        metrics=[keras.metrics.F1Score(average="macro", name="f1_score")])
    return model
return build
```

Zamena strategije pretrage je jedna linija — isti prostor, isti budžet, isto seme, pa je poređenje pošteno.

REZULTATI

Bajesova vs nasumična pretraga

Jedan model — MobileNetV2 — i tri varijante ulaza. Isti prostor pretrage, isti budžet od 25 pokušaja, isto seme. Pobednik po validacionom makro F1.

sirovi kadrovi	nasumična	Ceo kadar nosi i suprotan smer saobraćaja — signal je bučan.
ROI isečak — finalna linija	bajesova	Maskiranje ostavlja samo relevantne trake — signal postaje naučiv.
ROI bez letterbox-a	bajesova	Isti smer i bez očuvanja odnosa strana — nalaz nije artefakt jedne obrade.

Referentne osnove

Nulto pravilo — uvek light

tačnost 0,395



Nasumično po distribuciji klasa

tačnost 0,404



CNN klasifikator — MobileNetV2 + ROI

tačnost 0,870



Nasumično predviđanje po dristribuciji klasa pobeđuje nulto pravilo na makro F1, a gubi na tačnosti. Ta inverzija je razlog zašto je makro F1 glavna metrika, a tačnost prateća.

Matrica konfuzije

TEST SKUP · 1.411 SLIKA · PREDVIĐENO →

	no_traffic	light	high
no_traffic	681	68	1
light	59	460	38
high	0	18	86

0,833 0,870

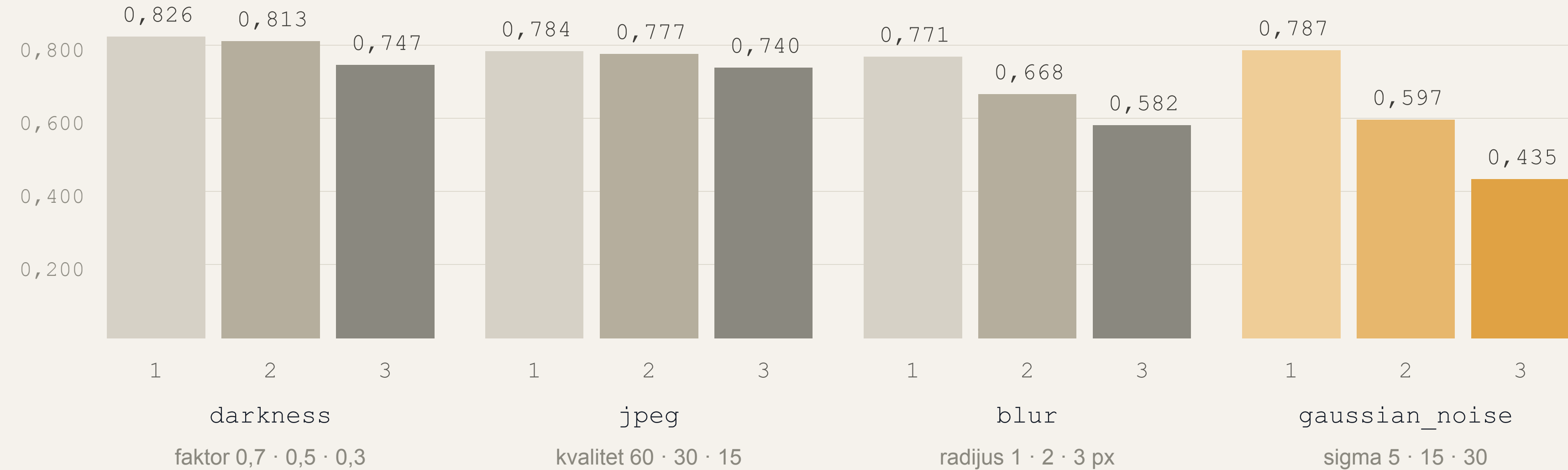
makro F1 tačnost

Greške su skoro isključivo između susednih nivoa gustine — jedan prazan prelaz nazvan gužvom, nula gužvi nazvanih praznim. Model ne pravi katastrofalnu zamenu.

Slaba klasa ostaje high , i to po preciznosti a ne po odzivu — posledica težinski ponderisane funkcije greške na retkoj klasi.

Perturbacioni testovi

y-osa · makro F1 na test skupu
— čist ulaz — 0,833



Mrak je skoro bezbolan — noćni snimci prolaze i na najvišoj jačini.

Gausov šum je najgori slučaj: na jačini 3 rezultat pada ispod polovine početne vrednosti.

Kod — perturbazioni testovi

src/evaluation/perturb.py

```
def perturb(arr: np.ndarray, family: str, severity: int) -> np.ndarray:
    if severity == 0:
        return arr

    if family == "gaussian_noise":
        sigma = {1: 5.0, 2: 15.0, 3: 30.0}[severity]
        rng = np.random.default_rng(0)
        return np.clip(arr + rng.normal(0, sigma, arr.shape), 0, 255)

    if family == "jpeg":
        quality = {1: 60, 2: 30, 3: 15}[severity]
        img = Image.fromarray(arr.astype(np.uint8))
        buf = io.BytesIO()
        img.save(buf, format="JPEG", quality=quality)
        buf.seek(0)
        return np.asarray(Image.open(buf).convert("RGB"),
dtype=np.float32)
    if family == "blur":
        radius = {1: 1.0, 2: 2.0, 3: 3.0}[severity]
        img = Image.fromarray(arr.astype(np.uint8))
        return np.asarray(img.filter(ImageFilter.GaussianBlur(radius)),
dtype=np.float32)

    if family == "darkness":
        factor = {1: 0.7, 2: 0.5, 3: 0.3}[severity]
        return np.clip(arr * factor, 0, 255)

    raise ValueError(family)
```

Treniranje i evaluacija — zaključci

- 01 Bajesova optimizacija pobeđuje nasumičnu pretragu.
- 02 Ulazna reprezentacija nosi najveći dobitak: ROI maskiranje je pomerilo rezultat jače nego bilo koje podešavanje klasifikacione glave unutar pretrage.
- 03 Agregatna metrika nije dovoljna, perturbacioni testovi i podela po kamerama otkrivaju slabosti koje jedna ujedinjena metrika potpuno sakriva.

Primena i serviranje modela

Post-training kvantizacija i model izložen kao REST endpoint u Docker kontejneru

ŠABLON PRIMENE

Dinamička primena na serveru, u kontejneru

IZABRANA OPCIJA

Tehnika kompresije — TFLite kvantizacija

REŽIM SERVIRANJA

Na zahtev — jedan zahtev, jedna predikcija

Post-training kvantizacija

VARIJANTA	VELIČINA	KAŠNJENJE P50	MAKRO F1	SAGLASNOST
Keras float32	24,57 MB	10,57 ms	0,809	—
TFLite float16	11,84 MB	7,48 ms	0,811	99,9%
TFLite dynamic-range	6,20 MB	4,24 ms	0,798	97,0%
TFLite int8 – servirano	6,41 MB	2,83 ms	0,801	92,8%

int8 je 74% manji i 3,7× brži, uz gubitak od 0,008 makro F1.
float16 je besplatan: 52% manji, bez merljive razlike.

Saglasnost od 92,8% znači da int8 menja svaku četrnaestu predikciju dok zadržava zbirni rezultat.

Kod — konverzija i kvantizacija

src/deployment/quantize.py



```
def to_tflite(saved_model_dir, mode, rep_gen=None):
    import tensorflow as tf
    conv = tf.lite.TFLiteConverter.from_saved_model(str(saved_model_dir))
    if mode == "float16":
        conv.optimizations = [tf.lite.Optimize.DEFAULT]
        conv.target_spec.supported_types = [tf.float16]
    elif mode == "dynamic":
        conv.optimizations = [tf.lite.Optimize.DEFAULT]
    elif mode == "int8":
        conv.optimizations = [tf.lite.Optimize.DEFAULT]
        conv.representative_dataset = rep_gen
        # float in/out with int8 internals: keeps the evaluation loop identical
        # across variants. Fully int8 I/O would need manual input quantization.
        conv.target_spec.supported_ops = [
            tf.lite.OpsSet.TFLITE_BUILTINS_INT8,
            tf.lite.OpsSet.TFLITE_BUILTINS
        ]
    else:
        raise ValueError(mode)
    return conv.convert()
```

REST API

POST /predict

Jedna slika unutra, klasa i verovatnoće kao izlaz.
FastAPI u Docker kontejneru, model
model_int8.tflite.

```
{  
  "camera": "BATROVCI_I",  
  "prediction": "no_traffic",  
  "confidence": 0.9961,  
  "latency_ms": 78.7  
}
```

src/serve/app.py

```
@app.post("/predict")  
@limiter.limit("60/minute")  
async def predict(request: Request, file: UploadFile =  
File(...), camera: str = Form(None),  
username: str = Depends(auth.current_user)):  
    raw = await _read_capped(file)  
    cam = camera or camera_dir_from_path(file.filename or "")  
    probs, latency_ms = _infer(raw, cam)  
    result = _result(cam, probs, latency_ms)  
    _record(result, username, filename=file.filename)  
    return result
```

Ograničenja

Klasa high na 104 slike

Široke granice greške na klasi koja dominira makro F1.
Male razlike među varijantama ne treba prečitavati.

Confidence nije kalibrisan

Prosečan confidence je 0,980 uz tačnost 0,878. Dovoljno za argmax, ali prag na verovatnoći bi čitao šum.

Nema prenosa na nov prelaz

Geometrija traka je specifična po kameri. Nova kamera se konfiguriše ROI poligonom, ne izvodi automatski.

Prisutan vremenski drift

Udeo klase high pada sa 18% na 7% između trening i test perioda. Očekuje se degradacija kroz vreme.

Hvala na pažnji

Pitanja?